T3000 API & Protocol

Centralized Logging Design · built Nov 13, 2026

Centralized Logging Design

> Final architecture doc for the policy-driven logging rollout. See centralized-database-design.md and centralized-database-multipc.md for related backend architecture.

1. Final Decision

This is the canonical documentation for the centralized logging module.

- Keep this file as the single source of truth.
- Do not keep a duplicate copy in `docs/todo/`.

2. Current State (Truth)

Centralized logging is implemented, but rollout is still hybrid:

- Centralized path is active via `LoggingService`.
- Some legacy direct `ServiceLogger` calls still exist.
- Centralized sinks currently in use: DB and file (policy-driven).
- Console output still appears from legacy paths and is not a centralized policy sink.

3. Unified Target Flow

All emitters should end up using one path:

- emitter -> `LoggingService` -> policy evaluation -> sink routing -> DB/file

Operational rules:

1. Category/level are normalized in one place.
2. Runtime category policy controls enable/min-level/detail/sinks.
3. Errors always persist to DB (safety invariant).
4. High-volume categories prefer MSSQL when available; fallback to SQLite.
5. Existing file naming behavior stays compatible.

4. What Is Already Done

- Added centralized module under `api/src/logging/`.
- Routed core app-log path through centralized service.
- Updated frontend legacy LogUtil to shared implementation.
- Aligned file-log endpoint base path in React logs page.
- Documentation updated to reflect hybrid rollout instead of "fully complete".

5. What Must Still Be Done (Required)

To reach full centralization, remaining legacy direct file logging must be migrated.

Priority cleanup scope:

1. FFI sync paths
2. Partition monitor/query paths
3. Other remaining `ServiceLogger` direct call sites

Expected final behavior after cleanup:

- All logs (backend + frontend-originated) follow centralized policy.
- File output still exists, but only through centralized emission.
- No split behavior between legacy direct file writes and policy-based writes.

6. Acceptance Checklist

1. Single Rust logging facade handles all log events.
2. Logs page settings consistently affect backend and frontend-originated logs.
3. Existing file naming remains compatible.
4. Activity Log categories/levels remain correct.
5. Error events persist even if category policy is restrictive.
6. No duplicate writes from mixed old/new paths.
7. File Logs page route/path remains aligned with backend endpoint.

7. Validation Snapshot

- `cargo check`: pass
- `cargo test --no-run`: environment-dependent (heavy build)

This is sufficient for a final docs commit. Full rollout validation continues during code migration.